

Documenting Architectures

Lecturer: Ngo Huy Bien
Software Engineering Department
Faculty of Information Technology
VNUHCM - University of Science
Ho Chi Minh City, Vietnam
nhbien@fit.hcmus.edu.vn

Objectives

- To *document* software architectures.



Contents

- I. Notations.
- II. Contents (Aspects).
- III. UML Architecture Diagrams.
- IV. The 4+1 View Model.
- V. The C4 Model.
- VI. Module Views.
- VII. Module Styles.
- VIII. Component-and-Connector (C&C) Views.
- IX. Component-and-Connector Styles.
- X. Allocation Views.
- XI. Allocation Styles.



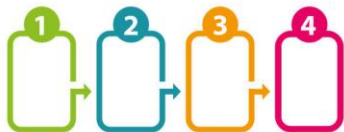
References

1. Simon Brown (2019). Software Architecture for Developers - Volume 2: Visualise, Document and Explore Your Software Architecture. Leanpub.
2. Paul Clements et al. (2010). Documenting Software Architectures - Views and Beyond. Pearson.

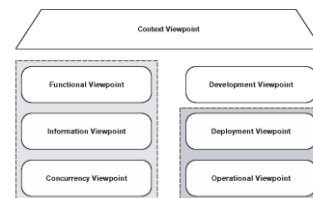


Notations

- Boxes and Arrows
- ADL
- UML

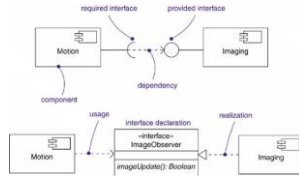


Contents (Aspects)



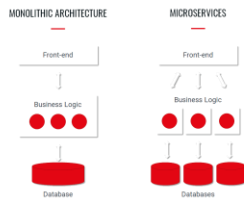
UML Architecture Diagrams

- I. Package Diagrams
- II. Component Diagrams
- III. Artifact Diagrams
- IV. Deployment Diagrams

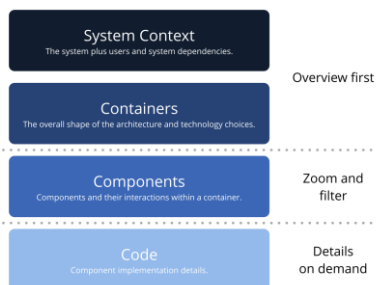


The 4+1 View Model Example

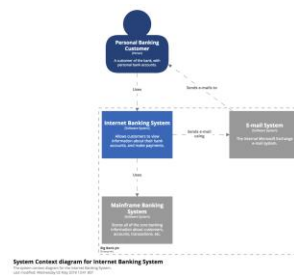
- From monolith to microservices.



The C4 Model [1]



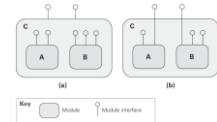
System Context





Module Views (I) [2]

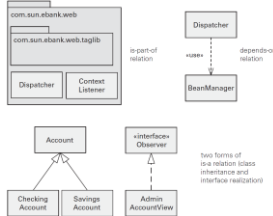
- **Modules** pertain to the way in which a system's software is decomposed into manageable units of responsibilities, which is one of the important forms of system structure.
- A module view provides a **blueprint** for the source code and the data model.
 - Name.
 - Responsibility.
 - Visibility of interface(s).
 - Implementation information.



(a) Module C provides its own interface, hiding the interfaces of modules A and B. (b) Module C exposes a subset of the interfaces of modules A and B as its interface.

Module Views (II)

- Modules are **related to one another** by forms of is-part-of, depends-on, and is-a relations.

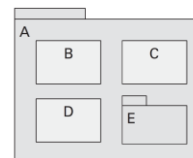


Module Styles

- The **decomposition style**, used to show the structure of modules and sub-modules (that is, containment relations among modules).
- The **uses style**, used to indicate functional dependency relations among modules.
- The **generalization style**, used to indicate specialization relations among modules.
- The **layered style**, used to describe the allowed-to-use relation in a restricted fashion between groups of modules called layers.
- The **aspects style**, used to describe particular modules called aspects that are responsible for crosscutting concerns.
- The **data model style**, used to show the relations among data entities.

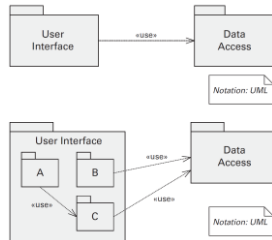
Decomposition Style

- A decomposition view shows how responsibilities are allocated across modules and sub-modules.
- In UML, module decomposition is shown by nesting, with the aggregate module shown as a package.



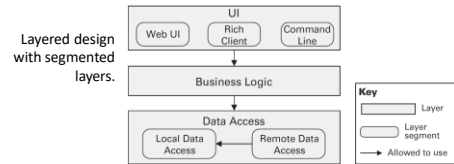
Uses Style

- A uses view shows how modules depend on one another.
- This view helps achieve incremental development and is especially suitable for performing change-impact analysis.
- The User Interface module has been decomposed into modules A, B, and C.
- At least one of the modules must depend on the Data Access module or the decomposition would not be consistent.



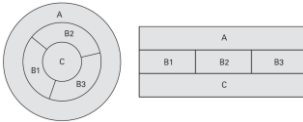
Layered Style (I)

- A **layered view** divides a system into groups of modules that provide cohesive responsibilities.
- These groups are called **layers** and relate to each other uni-directionally by the allowed-to-use relation.
- A layered design helps a system achieve **portability** and **modifiability**.



Layered Style (II)

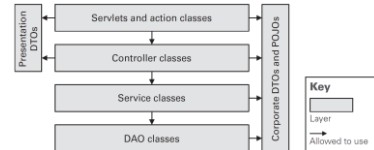
- A layered design shown as concentric rings and as a stack of boxes. Are these two representations equivalent?



- In the layer diagram, B1 and B3 are separate; in the ring diagram they are adjacent.

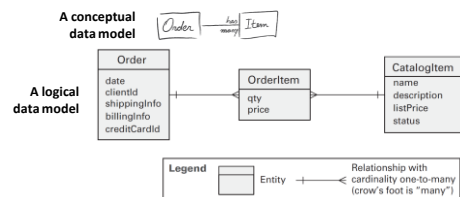
Layered Style (III)

- Layers with a "sidcar."
- The key should make clear what is allowed to use and be used by software in the box on the side.



Data Model View

- A **data model view** describes the structure of the data used in the system in terms of data entities and their relationships.
- It guides implementation and helps to improve **performance** and **modifiability** in data-centric systems.



Component-and-Connector Styles

- A **component-and-connector (C&C) style** introduces a specific set of component-and-connector types and specifies rules about how elements of those types can be combined.
- Additionally, given that C&C views capture **runtime aspects** of a system, a C&C style is typically also associated with a **computational model** that prescribes how data and control flow through systems designed in that style.

Pipe-and-Filter Style

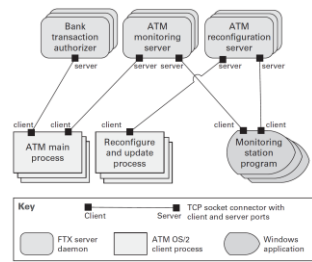
- Pipes** connect **filter output** ports to **filter input** ports.
- Connected filters must agree on **the type of data** being passed along the connecting pipe.
- Specializations** of the style may restrict the association of components to an **acyclic graph** or a linear sequence—sometimes called a pipeline.
- Other specializations** may prescribe that components have certain **named ports**, such as the **stdin**, **stdout**, and **stderr** ports of UNIX filters.

Why Pipe-and-Filter Style?

- Systems conforming to a pipe-and-filter style are typically used in **data transformation systems**, where the overall processing can be broken down into a set of **independent steps**, each responsible for an incremental transformation of its input data.
- The **independence** of the processing done by each step supports **reuse**, **parallelization**, and **simplified reasoning** about overall behavior.

Client-Server Style

- Clients** are connected to **servers** through **request/reply** connectors.
- Server components can be clients to other servers.
- Specializations may impose restrictions:
 - Numbers of attachments to a given port
 - Allowed relations among servers

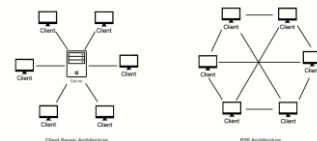


Why Client-Server Style?

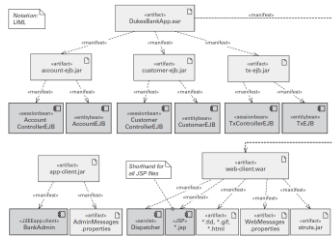
- The client-server style presents a system view that separates **client applications** from **the services** they use.
- This style supports system understanding and **reuse** by factoring out **common services**.
- Because servers can be accessed by any number of clients, it is relatively easy to add **new clients** to a system.
- Similarly, servers may be **replicated** to support scalability or availability.

Peer-to-Peer Style

- Restrictions may be placed on the number of allowable attachments to any given port, or role.
- Special peer** components can provide **routing**, **indexing**, and **peer search capability**.
- Specializations may impose visibility restrictions on which components can know about other components.



Example Install View



Example Work Assignment View

ECS Element (Module)		
Segment	Subsystem	Organizational Unit
Science Data Processing Segment (SDPS)	Client	Science team
	Interoperability	Prime contractor team 1
	Ingest	Prime contractor team 2
	Data Management	Data team
	Data Processing	Data team
	Data Server	Data team
	Planning	Orbital vehicle team
Flight Operations Segment (FOS)	Planning and Scheduling	Orbital vehicle team
	Data Management	Database team
	User Interface	User interface team

Further Reading

- Lenny Delligatti (2013). SysML Distilled: A Brief Guide to the Systems Modeling Language. Addison-Wesley Professional.



Thank You & See You Again

